
E-Commerce Platform Architecture & Development Documentation

Table of Contents

- 1. Introduction
- 2. System Architecture Diagram
- 3. User Journey and Application Workflow
- 4. Database Schema and Entity-Relationship Diagram
- 5. Backend Architecture and APIs
- 6. Frontend Architecture (React & Flutter)
- 7. Security Model
- 8. Payment Flow Diagram
- 9. Admin Dashboard Functionalities
- 10. CI/CD Pipeline and Deployment Diagram
- 11. Performance and Scalability
- 12. Compliance and Data Privacy
- 13. Appendix: Code Samples & API Specs

1. Introduction

This document provides a detailed overview of the full-stack e-commerce platform built with ReactJS and Flutter frontends, a Python Django backend, and PostgreSQL for data storage. It explains system design, database models, security, API definitions, and developmental best practices to guide implementation, testing, and deployment.

2. System Architecture Diagram

text

[User Devices]

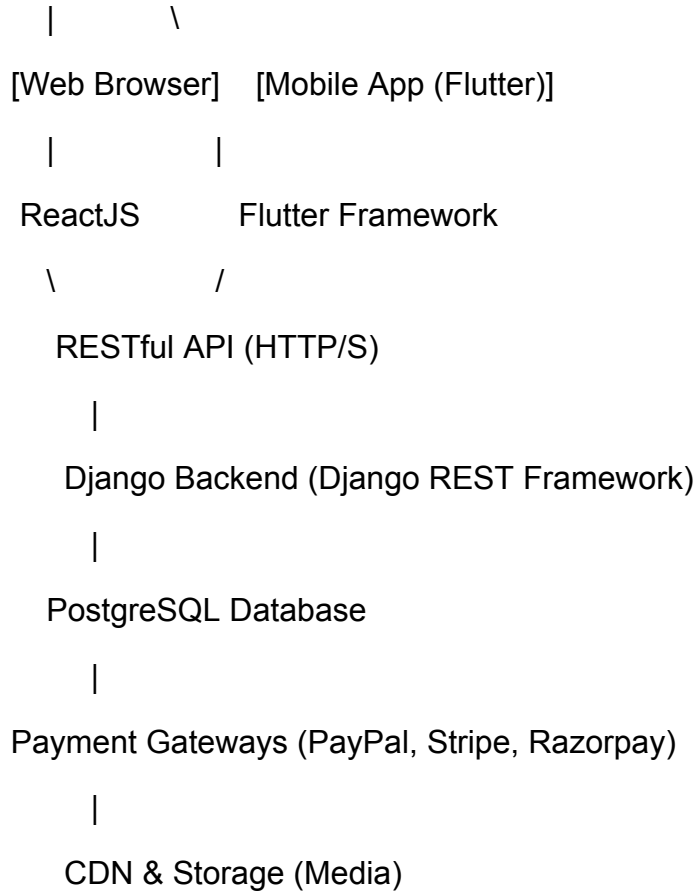


Diagram Description:

Users access the platform either via a React web app or a Flutter mobile app. Both frontends communicate securely over REST APIs to the Django backend. The backend manages business logic and data persistence in PostgreSQL. Payment gateways interact with the backend via server-side APIs and webhooks. Media assets are served through CDN/S3.

3. User Journey and Application Workflow

User Journey Stages

1. **Landing & Browsing**

User accesses the homepage, browses product categories, applies filters, views product details.

2. **Account Management**

User registers/logs in, manages profile and shipping addresses.

3. **Cart & Checkout**

User adds items to the cart, edits quantities, proceeds to checkout, chooses payment.

4. **Payment Processing**

Secure payment processing via integrated gateways with webhook confirmation.

5. **Order Fulfillment**

Order status updates: Processing → Shipped → Delivered.

6. **Post-Purchase Interaction**

Users can leave reviews, request refunds, or reorder.

Workflow Diagram:

text

flowchart TD

A[Browse Products] --> B[Add to Cart]

B --> C{User Logged In?}

C -- Yes --> D[Checkout]

C -- No --> E[User Login/Register]

E --> D

D --> F[Payment Gateway]

F --> G{Payment Success?}

G -- Yes --> H[Order Confirmation]

G -- No --> I[Payment Failure]

H --> J[Order Processing & Shipping]

J --> K[Delivery]

K --> L[User Reviews & Feedback]

4. **Database Schema and Entity-Relationship Diagram**

Key Entities and Relationships

- **User (1) —> (Many) Address**

- **Category** (1) —> (Many) **Product**
- **Product** (1) —> (Many) **ProductImage**
- **User** (1) —> (Many) **Cart** (Only one active)
- **Cart** (1) —> (Many) **CartItem**
- **User** (1) —> (Many) **Order**
- **Order** (1) —> (Many) **OrderItem**
- **Order** (1) —> (Many) **Payment**
- **User** (1) —> (Many) **Review** on **Product**

Entity-Relationship Diagram (ERD):

text

erDiagram

USER ||--o{ ADDRESS : has

USER ||--o{ CART : owns

CART ||--o{ CARTITEM : contains

PRODUCT ||--o{ PRODUCTIMAGE : includes

CATEGORY ||--o{ PRODUCT : groups

USER ||--o{ ORDER : places

ORDER ||--o{ ORDERITEM : consists_of

ORDER ||--o{ PAYMENT : processed_by

USER ||--o{ REVIEW : writes

PRODUCT ||--o{ REVIEW : receives

5. Backend Architecture and APIs

Django Backend Modules

- **Authentication:** JWT-based login/registration with role-based access (customer/admin).

- **Product Catalog:** APIs for browsing, searching, filtering products.
- **Cart Management:** Add/update/delete items, manage persistent carts.
- **Order Processing:** Create orders, update statuses, history.
- **Payments:** Integration with multiple providers; webhook handling for async payment confirmation.
- **User & Profile:** Addresses, profile data management.
- **Reviews:** User-generated feedback moderated and rate-limited.

Example API Endpoint for Products

text

GET /api/products/?category=12&search=shirt&sort=price_asc

Authorization: Bearer <token>

Response:

```
[
  {
    "id": 123,
    "name": "Cotton Shirt",
    "price": "29.99",
    "currency": "USD",
    "slug": "cotton-shirt",
    "category": "Shirts"
  },
  ...
]
```

6. Frontend Architecture

React Web App

- Component-based structure using React Router, Redux/store for state.
- Responsive design with Bootstrap or Tailwind.
- Token management via HttpOnly cookies.
- API communication using Axios with interceptors for refreshing JWT.
- Form validation and error handling.

Flutter Mobile App

- MVVM pattern with Provider or Riverpod for state management.
 - Secure token storage using flutter_secure_storage.
 - Rich UI widgets for navigation, forms, and lists.
 - Offline handling with local cache.
 - API interaction via Dio or HTTP packages.
 - Push notifications incorporation.
-

7. Security Model

- Passwords salted and hashed with Argon2/PBKDF2.
 - Tokens stored securely with short expiration.
 - Backend enforces authorization checks strictly on every API.
 - CSRF protection on state-changing requests.
 - User input sanitized to prevent XSS and SQL Injection.
 - File uploads validated and served via signed URLs.
 - Payment flow secured by webhook signature verification and idempotency keys.
 - Rate limiting applied on login, signup, cart edits, and payments.
-

8. Payment Flow Diagram

text

sequenceDiagram

participant User

participant Frontend

participant Backend

participant PaymentGateway

User->>Frontend: Initiates checkout

Frontend->>Backend: Request payment intent

Backend->>PaymentGateway: Create payment session

PaymentGateway-->>Backend: Returns session info

Backend-->>Frontend: Sends session data

Frontend->>PaymentGateway: Redirect/payment widget

PaymentGateway-->>Backend: Webhook payment result

Backend-->>Frontend: Update order status + confirmation

Frontend->>User: Show payment success/failure

9. Admin Dashboard Functionalities

- Product CRUD with bulk upload capability.
- Category management including hierarchy.
- Order tracking and manual status update.
- Payment report reconciliation.
- User management with role assignments and activity logs.
- Analytics: Sales over time, popular products, customer activity, abandoned carts.
- Security and audit trails for admin actions.

10. CI/CD Pipeline and Deployment Diagram

text

[Developer Push] --> [CI Pipeline: Tests & Lint] --> [Build Docker Images / Web Bundles]
--> [Deploy to Staging] --> [Manual/Auto Approval] --> [Deploy to Production]

- Use GitHub Actions / GitLab CI for automation.
 - Dockerized backend services.
 - Frontend deployment on Netlify/Vercel or S3 + CloudFront.
 - Database migrations automated during deployment.
 - Monitoring hooks integrated with alerting.
-

11. Performance and Scalability

- Use CDN for static assets and media.
 - Database indexed for all FK relations and frequently queried columns.
 - Use Django ORM prefetch/select_related to avoid N+1 query issues.
 - Separate read-replica database for heavy catalog queries.
 - Caching layers (Redis) for session and frequently accessed catalog data.
 - Horizontal scaling with load balancers.
-

12. Compliance and Data Privacy

- Adherence to GDPR, CCPA guidelines.
 - Support for user data export and deletion requests.
 - PCI-DSS compliant payment integration (delegated to third parties).
 - Encryption in transit (TLS) and at rest.
 - Minimal PII storage and strict access controls.
-

13. Appendix: Code Snippets & API Specs

- Sample Django model for Product with validations.

- [React sample for secure login flow.](#)
 - [Flutter example for cart state management.](#)
 - [Swagger/OpenAPI auto-generated docs link.](#)
-